Below is a **complete master table** of **Java Exception**, its **types**, **difference between Checked vs Unchecked vs Error**, **when to use what**, and **how exceptions work with method Overloading & Overriding** (same class + subclass) with examples.

✅ 1) Java Exception Hierarchy (Types of Exceptions)

Master Table (Throwable → Exception / Error)

Level	Class/Type	Category	Checked?	Raised by	Can we handle?	Example
1	Throwable	Root class	-	JVM / Program	✅ Yes	Base of all exceptions/errors
2	Exception	Application-related problems	✅ Mostly Checked	Program	✅ Yes	IO issues, DB issues
3	Runtime Exception	Programming mistakes	❌ Unchecked	Program	✅ Yes	NullPointer, Arithmetic
2	Error	JVM/System failure	❌ Not meant to catch	JVM	❌ Avoid catching	OutOfMemoryError

---

✅ 2) Checked vs Unchecked vs Error (Full Comparison Table)

Feature	Checked Exception	Unchecked Exception	Error
Parent Class	Exception (excluding RuntimeException)	RuntimeException	Error
Compiler Check	✅ Yes (must handle)	❌ No	❌No
Must use try-catch or throws	✅ Yes	❌ Not mandatory	❌Not recommended
Indicates	Recoverable issue (external)	Programming bug	JVM/System failure
When occurs	Compile-time awareness	Runtime only	Runtime (serious)

Best usage	File/DB/Network/API failures	Validate logic / fix code	Do not handle, fix system
Example	IOException, SQLException, ClassNotFound Exception	NullPointerException, ArithmeticException, IndexOutOfBounds Exception	OutOfMemoryError, StackOverflowError, VirtualMachineError
Should we catch?	✅ Often yes	✅ Sometimes (rare)	❌ Avoid

✅ 3) When to Use What? (Real Time Usage Table)

Situation	Use Checked?	Use Unchecked?	Use Error?	Example
File not found	✅ Yes	❌	❌	FileNotFoundException
DB connection failed	✅ Yes	❌	❌	SQLException
Wrong user input	❌ (better use validation)	✅ Yes	❌	IllegalArgument Exception
Null data received unexpectedly	❌	✅ Yes	❌	NullPointerException
Divide by zero bug	❌	✅ Yes	❌	ArithmeticException
Memory full / JVM crash	❌	❌	✅ Yes	OutOfMemoryError
Recursive infinite call	❌	❌	✅ Yes	StackOverflowError

✅ 4) How Exceptions Work in Java (Flow Table)

Step	What Happens
1	Exception occurs in code
2	JVM creates an Exception object
3	JVM checks whether there is a matching try-catch
4	If found → control jumps to catch block
5	If not found → method terminates, exception is propagated to caller
6	If reaches main and still not handled → program terminates + stack trace printed
7	finally executes always (except System.exit / JVM crash)

✅ 5) Exception Rules with Overloading and Overriding

✅ A) Exception with Method Overloading (Same Class)

✅ Rule Table: Overloading & Exceptions

Rule	Allowed?	Explanation
Overloaded methods can throw different checked exceptions	✅ Yes	Each method is treated separately
Overloaded method can throw broader exceptions	✅ Yes	No restriction like overriding
Overloaded method can throw unchecked exceptions	✅ Yes	Unchecked never restricted
Compile-time selection matters	✅ Yes	Overloading depends on reference + method signature

---

✅ Example: Overloading in Same Class

```
import java.io.*;

class A {

   // Checked exception
   void readFile() throws IOException {
      FileInputStream fis = new FileInputStream("abc.txt");
   }

   // Overloaded method - can throw different checked exception
   void readFile(String path) throws FileNotFoundException {
      FileInputStream fis = new FileInputStream(path);
   }

   // Overloaded method - unchecked exception
   void readFile(int num) throws ArithmeticException {
      int x = 10 / num;
   }
}
```

✅ **Conclusion (Overloading):**
Overloading has **NO strict exception restriction**.

✅ B) Exception with Method Overriding (Subclass)

✅ Rule Table: Overriding & Exceptions

Rule	Allowed?	Explanation
Overriding method can throw same exception	✅ Yes	Same as parent method
Overriding method can throw subclass exception	✅ Yes	Narrower exception allowed
Overriding method can throw broader checked exception	❌ No	Compiler error
Overriding method can throw new checked exception	❌ No	Not declared in parent
Overriding method can throw unchecked exception	✅ Yes	RuntimeException allowed always
If parent throws no checked exception, child cannot throw checked exception	❌ No	Compiler error

---

✅ Example: Overriding (Subclass)

```
import java.io.*;

class Parent {
   void show() throws IOException {
      System.out.println("Parent show()");
   }
}

class Child extends Parent {

   // ✅ allowed: narrower exception
   @Override
   void show() throws FileNotFoundException {
      System.out.println("Child show()");
   }

   // ✅ allowed: unchecked (runtime)
   void test() throws NullPointerException {
      System.out.println("Runtime exception allowed");
   }

   /*
```

```
   ❌ Not allowed: broader checked exception
   void show() throws Exception { }
   */
}
```

✅ 6) Same Program: Overloading + Overriding Combined Example (Full)

✅ Example: same class + subclass combined

```
import java.io.*;

class Parent {

   // Parent method throws Checked IOException
   void m1() throws IOException {
      System.out.println("Parent m1()");
   }

   // Overloading in parent (different signature)
   void m1(int x) throws FileNotFoundException {
      System.out.println("Parent m1(int)");
   }
}

class Child extends Parent {

   // ✅ Overriding: can throw same or narrower checked exception
   @Override
   void m1() throws FileNotFoundException {
      System.out.println("Child m1()");
   }

   // ✅ Overloading in child (new method)
   void m1(String s) throws Exception {
      System.out.println("Child m1(String)");
   }
}

public class ExceptionDemo {
   public static void main(String[] args) {
      Parent p = new Child();

      try {
         p.m1();        // calls Child m1() (override) at runtime
         p.m1(10);      // calls Parent m1(int) (not overridden)
      } catch (IOException e) {
         System.out.println("Handled: " + e);
      }
```

```
    }
}
```

✅ Key Point: Overloading vs Overriding (Exception Behavior)

Feature	Overloading	Overriding
Decided at	Compile time	Runtime
Exception restriction	❌ No restriction	✅ Restriction for Checked exceptions
Can throw broader checked exception	✅ Yes	❌ No
Can throw unchecked exception	✅ Yes	✅ Yes

✅ 7) Quick Interview Notes (Very Important)

✅ 1-line summary table

Type	Summary
Checked	"Recoverable + compiler forces handling"
Unchecked	"Programming mistake + no compiler force"
Error	"JVM failure; don't catch; fix infra/code recursion/memory"

If you want, I can also provide a **single combined Exception + equals/hashcode + String + collections master mega-table** for interview revision (0–3 years).